ТЗ: игровое расширение для VS Code (Kahoot + Codewars в классе)

Nov 10, 2026 · @qwerty

Обзор проекта

Расширение для VS Code, которое превращает урок программирования в интерактивную игру: учитель создаёт комнату, ученики входят по ссылке или коду, и все вместе играют в пять режимов на тему, которую задал учитель. По духу это Kahoot плюс Codewars, но внутри редактора, где ученики и так пишут код.

Рабочее название: CodeArena (можно поменять).

Для кого:

- **Учитель (хост)** создаёт комнату, делит учеников на команды, задаёт тему или свои задания, запускает игры и видит всё, что происходит.
- **Ученик** ставит расширение, вводит своё имя, играет и видит результаты.

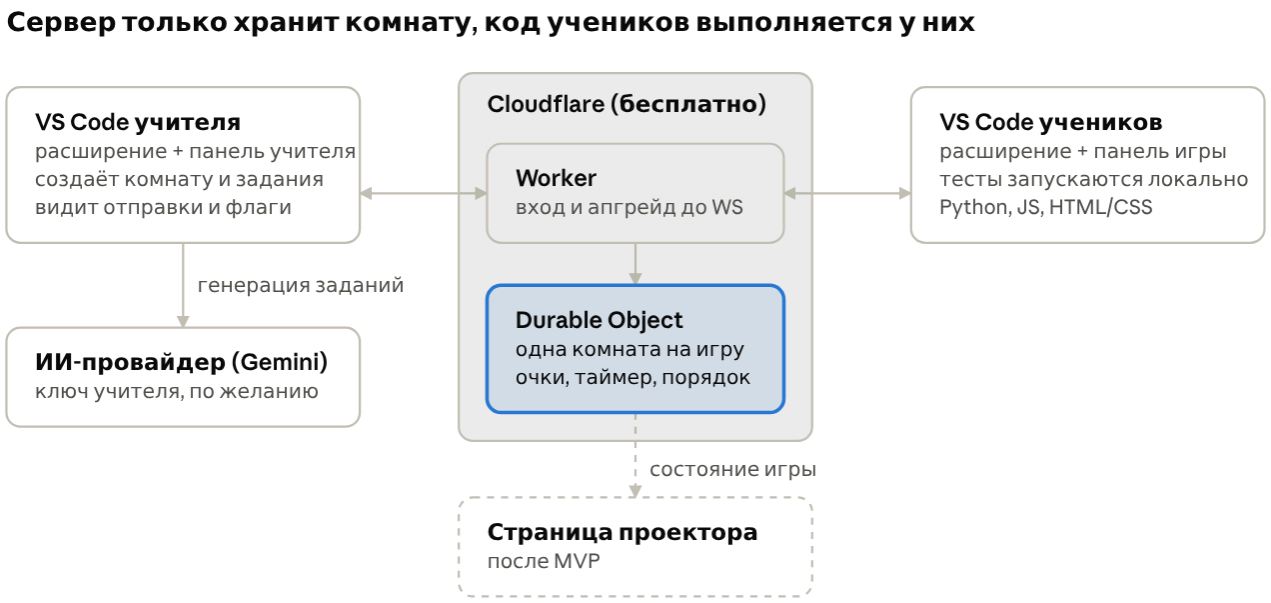
Жёсткие ограничения:

- **Open source**, лицензия MIT, публичный репозиторий на GitHub.
- **Ноль расходов.** Проект не зарабатывает и не тратит деньги: только бесплатные тарифы и ресурсы самих пользователей.
- **Работает через интернет**, а не только в одной Wi-Fi сети.
- **Красивый и понятный интерфейс** — главный приоритет проекта. Ученик должен понять, что делать, за 5 секунд без инструкции.
- **Языки интерфейса:** русский и кыргызский, английский по умолчанию для open source.
- **Языки задач в первой версии:** Python, JavaScript, HTML/CSS.

Архитектура и стек

Монорепозиторий на TypeScript из четырёх пакетов. Сервер — бесплатный Cloudflare Workers + Durable Objects, по одной «комнате» на игру. Код учеников выполняется локально на их компьютерах, поэтому сервер ничего не запускает и остаётся бесплатным.

Пакет	Что внутри	Стек
`extension`	Расширение VS Code: вход, команды, файлы задач, запуск тестов, анти-чит	TypeScript, VS Code Extension API
`webview-ui`	Весь интерфейс игры внутри VS Code (панели ученика и учителя)	React, Vite, Tailwind CSS, Framer Motion
`server`	Комнаты, игроки, команды, очки, таймеры, проверка ответов	Cloudflare Workers, Durable Objects (SQLite, бесплатный план), WebSocket Hibernation API
`shared`	Общие типы, протокол сообщений, схемы заданий	TypeScript, Zod



архитектура · учитель, Cloudflare, ученики

Учитель и ученики держат постоянное WebSocket-соединение с Worker'ом, тот передаёт его в Durable Object комнаты. ИИ вызывается напрямую из расширения учителя, сервер в этом не участвует.

Дополнительно: страница для проектора (тот же React, раздаётся с Worker'а) — по желанию, после MVP.

Где что проверяется:

- **Ответы-значения** (канат, викторина, «найди строку», «выбери лучший вариант») проверяет сервер. Правильный ответ хранится только на сервере и никогда не отправляется ученикам.

- **Код** (Codewars, «исправь баг», задачи гонки с кодом) проверяется локально в расширении ученика, на сервер уходят код и результат тестов.
  - Python: `python3` ученика через дочерний процесс, тайм-аут 5 секунд.
  - JavaScript: отдельный `worker_thread` в Node, тайм-аут 5 секунд.
  - HTML/CSS: парсинг через `parse5` или `node-html-parser` и проверка правил по CSS-селекторам.
- Если Python не установлен, расширение показывает понятное сообщение с ссылкой на установку.

Лимиты бесплатного Cloudflare: 100 000 запросов в день, входящие WebSocket-сообщения считаются 20 к 1, при превышении операции падают с ошибкой, а не выставляется счёт. Для нескольких классов в день хватает с запасом. Для школ, которым мало, — Docker-образ для самостоятельного запуска сервера (после MVP).

Вход, имена, лобби и команды

Ученик входит без GitHub и без регистрации: ссылка или код, затем своё имя, и он в лобби. Учитель видит всех и сам решает, кто в какой команде.

Вход:

1. Учитель нажимает «Создать комнату» и получает 6-значный код и ссылку вида `vscode://<publisher>.codearena/join?code=482913`. Ссылку он кидает в группу.
2. Ученик открывает ссылку (VS Code откроется сам) или вводит код в панели расширения вручную.
3. Ученик пишет имя и фамилию или первую букву фамилии. Имя сохраняется локально, в следующий раз поле уже заполнено.
4. Если такое имя уже есть в комнате, ученика просят дописать фамилию.

Переподключение: при первом входе сервер выдаёт токен, расширение хранит его до конца игры. Если пропал интернет или закрылся VS Code, ученик возвращается в ту же команду со своими очками, а не как новый игрок.

Лобби учителя:

- Список подключившихся в реальном времени, счётчик «В комнате: 27».
- Режим деления: каждый сам за себя, 2 команды или 3 команды.
- Деление вручную перетаскиванием имён между колонками команд, или кнопка «Распределить случайно» с последующей ручной правкой.

- У каждой команды цвет и название (по умолчанию «Красные», «Синие», «Зелёные»), учитель может переименовать.
- Учитель может переименовать ученика (если тот написал «Бэтмен») и выгнать из комнаты.
- Опоздавший попадает в лобби ожидания, учитель добавляет его в команду одним кликом.

Лобби ученика: своё имя, цвет и состав своей команды, анимированное ожидание «Ждём, пока учитель начнёт игру».

Пять игр

В первой версии пять режимов. Все, кроме каната, работают в любом делении: каждый сам за себя, 2 или 3 команды.

Игра	Тип задания	Деление	Как побеждают
Перетягивание каната	Короткий ответ (число, слово, строка)	Только 2 команды	Канат дошёл до края или у кого перевес к концу времени
Викторина	Вопрос с 2–4 вариантами	Любое	Больше всех очков
Codewars	Код с тестами	Любое	Место в списке правильных решений
Гонка	Серия задач любого типа	Любое	Первым дошёл до финиша
Найди баг	Строка, исправление или выбор варианта	Любое	Больше всех очков

1. Перетягивание каната

- Обе команды одновременно получают одну и ту же задачу (например, `5 + 8` или «что выведет `print(len('hello'))`»).
- Кто первым прислал правильный ответ, тот тянет канат на 1 шаг в свою сторону. Порядок определяет сервер по моменту прихода сообщения, а не время на компьютере ученика. Durable Object обрабатывает сообщения строго по очереди, поэтому двух «первых» не бывает.
- Сразу после правильного ответа обе команды получают следующую задачу.

- Неправильный ответ блокирует ввод этому ученику на 3 секунды (учитель может менять от 0 до 10), чтобы не угадывали спамом.
- Канат — шкала от −5 до +5. Победа: канат дошёл до края или по окончании времени (по умолчанию 3 минуты) перевес у одной из команд.
- Для 3 команд эта игра недоступна, интерфейс предлагает гонку.

2. Викторина (как Kahoot)

- Вопрос с 2–4 вариантами ответа, у каждого варианта свой цвет и фигура, таймер на вопрос (по умолчанию 20 секунд).
- Очки: правильно = от 500 до 1000, чем быстрее, тем больше. Формула: `1000 × (1 − время_ответа / лимит / 2)`. Неправильно или не успел = 0.
- После каждого вопроса: правильный ответ, сколько человек выбрали каждый вариант, топ-5.
- В командах очки команды = среднее по её участникам, чтобы большая команда не выигрывала числом.

3. Codewars

- Ученик получает задачу и файл с заготовкой функции, пишет код в обычном редакторе VS Code и жмёт «Отправить».
- Расширение прогоняет тесты локально. Если хоть один тест упал, ученик видит, какой именно, и может исправить и отправить снова.
- Неправильные решения скрыты от учеников, но учитель видит все.
- Правильные решения выстраиваются списком:
  1. Короче — выше. Длина считается без пробелов, табов и пустых строк, чтобы отступы не влияли.
  2. При одинаковой длине выше тот, кто отправил раньше.
- После окончания раунда все видят список правильных решений с кодом, как в Codewars, и могут поучиться у других.
- В командах очки команды = число её участников с правильным решением, место в списке остаётся личным.
- Никакого голосования, только длина и время.

4. Гонка

- Серия из N задач (по умолчанию 10) любого типа: короткий ответ, вариант, код.
- Каждый правильный ответ двигает ученика или команду на 1 шаг к финишу. Неправильный ничего не даёт.

- Побеждает тот, кто первым дошёл до финиша, остальные места по позиции на момент окончания.
- В командах команда двигается на каждый правильный ответ любого участника.
- Подходит как замена каната для 3 команд.

5. Найди баг

Три типа заданий, учитель может смешивать их в одной игре. После каждого задания показывается короткое объяснение: почему это баг или почему этот вариант лучше.

- **Найди строку.** Показан код, ученик кликает на строку с ошибкой. Очки как в викторине: за скорость. Пример: `=` вместо `==`, забытый `return`, незакрытый тег.
- **Исправь баг.** Ученик получает сломанный код в редакторе, исправляет и отправляет.
  - Python и JavaScript проверяются тестами.
  - HTML и CSS проверяются правилами по селекторам: «есть `footer`», «у каждого `img` есть `alt`», «нет `div.footer`».
- **Выбери лучший вариант.** Показаны 2–4 рабочих варианта кода, один лучше. Правильный задан заранее, проверка автоматическая. Примеры: `<div class="footer">` или `<footer>`, цикл с `append` или list comprehension, `float` или `flexbox`.

Задания

Задания приходят из трёх источников: учитель пишет сам, ИИ генерирует по теме, или берутся из открытого банка задач. Все хранятся в одном формате JSON со схемой в пакете `shared` (Zod).

Типы заданий:

Тип	Поля	Где проверяется
`short_answer`	вопрос, правильные ответы (список допустимых), чувствительность к регистру	Сервер
`choice`	вопрос, 2–4 варианта, индекс правильного, объяснение	Сервер

Тип	Поля	Где проверяется
`code`	условие, язык, заготовка, видимые тесты	Расширение ученика
`find_line`	код, язык, номер строки с багом, объяснение	Сервер
`fix_bug`	сломанный код, язык, тесты или HTML-правила, объяснение	Расширение ученика
`best_variant`	2–4 фрагмента кода, индекс лучшего, объяснение	Сервер

У каждого задания ещё есть тема, сложность (лёгкая, средняя, сложная) и лимит времени.

Пример задания `code`:

```
{
  "type": "code",
  "topic": "Списки в Python",
  "difficulty": "easy",
  "language": "python",
  "prompt": "Верни список только чётных чисел из nums.",
  "starter": "def evens(nums):\n    pass",
  "tests": [
    { "call": "evens([1, 2, 3, 4])", "expected": "[2, 4]" },
    { "call": "evens([])", "expected": "[]" }
  ],
  "timeLimitSec": 300
}
```

Создание вручную: в панели учителя форма под каждый тип, с предпросмотром «как увидит ученик». Набор заданий можно сохранить в файл и загрузить снова.

Генерация через ИИ:

1. Учитель вводит тему («циклы for в Python», «семантические теги HTML»), выбирает игру, количество и сложность.
2. Запрос идёт к ИИ с ключом самого учителя (по умолчанию Google Gemini, у него есть бесплатный тариф). Ключ хранится в `SecretStorage` VS Code и никуда не отправляется, кроме API провайдера.
3. ИИ возвращает JSON, расширение проверяет его по схеме Zod. Невалидные задания отбрасываются и генерируются заново.

4. Учитель видит список сгенерированных заданий и может править, удалять, перегенерировать по одному, прежде чем запустить игру.
5. Для `code` и `fix_bug` расширение заранее прогоняет эталонное решение от ИИ через тесты и отбраковывает задания, где тесты не проходят.

Без ключа генерация недоступна, но всё остальное работает.

Открытый банк задач: папка `tasks/` в репозитории, задания в JSON по темам. В панели учителя кнопка «Взять из банка», любой учитель может прислать свои задания пулл-реквестом.

Возможности учителя

Учитель управляет всей игрой из одной панели в VS Code и видит в реальном времени, кто что делает.

До игры:

- Создать комнату, скопировать ссылку и код.
- Видеть лобби, делить на команды, переименовывать и выгонять (раздел «Вход, имена, лобби и команды»).
- Выбрать игру из пяти.
- Подготовить задания: вручную, через ИИ по теме или из банка.
- Настроить правила: время раунда, штраф за ошибку в канате, число задач в гонке, какие защиты от списывания включены.

Во время игры:

- Старт, пауза, следующий вопрос или задача, досрочное завершение.
- Живая таблица: каждый ученик, его статус (думает, отправил, решил, ошибся), очки, флаги анти-чита.
- Клик по ученику открывает его последнее отправленное решение, включая неправильные. Ученики чужие неправильные решения не видят.
- Счёт команд и визуализация игры (канат, трек гонки) такие же, как у учеников.

После игры:

- Итоговая таблица: места, очки, решённые задачи, флаги.
- Экспорт результатов в CSV для журнала.
- Перезапуск той же игры с теми же командами или новая игра без пересоздания комнаты.

Комната живёт, пока учитель её не закроет, и автоматически удаляется через 6 часов бездействия.

Защита от списывания

Защита не банит автоматически: она блокирует большие вставки и показывает учителю флаги, а решение принимает учитель. Каждую защиту учитель включает или выключает в настройках комнаты.

Защита	Как работает	Что видит учитель
Блок вставки	В файлах игры расширение перехватывает `Ctrl+V` / `Cmd+V` своей командой и разрешает вставку до 30 символов	Флаг «Вставка», число попыток
Детектор больших вставок	`onDidChangeTextDocument`: если одним изменением добавилось больше 30 символов (вставка через меню, ИИ-автодополнение), изменение откатывается	Флаг «Вставка», размер текста
Уход из окна	`window.onDidChangeWindowState`: VS Code потерял фокус	Флаг «Ушёл из окна», сколько раз и на сколько секунд
ИИ-расширения	При входе проверяется, установлены ли Copilot и другие ИИ-ассистенты из списка	Флаг «ИИ-расширение» с названием

- Порог 30 символов учитель может менять.
- Флаги видны только учителю, у ученика лишь короткое уведомление «Вставка отключена в этой игре».
- Защита не ловит телефон, второй компьютер и ручную перепечатку. Это нормально: таймер и соревнование сами мешают списывать.

Дизайн и интерфейс (главный приоритет)

Интерфейс должен ощущаться как Kahoot по энергии и как Linear или Vercel по чистоте: ярко в моменты игры, спокойно и аккуратно в остальное время. Если выбор между «больше функций» и «красивее и понятнее», выбираем второе.

Принципы

1. **Одно главное действие на экране.** Одна большая яркая кнопка (например, «Отправить» или «Начать игру»), всё остальное вторично.
2. **Понятно за 5 секунд.** Ученик без инструкции понимает, что делать. Никаких подписей мелким шрифтом, которые надо читать.
3. **Мгновенная обратная связь.** Каждое действие сразу отвечает анимацией: правильно — зелёная вспышка и звук, неправильно — лёгкая тряска поля, отправлено — галочка.
4. **Родной для VS Code.** Базовые цвета фона, текста и рамок берутся из темы VS Code (`--vscode-editor-background` , `--vscode-foreground` и т.д.), поэтому всё красиво и в светлой, и в тёмной теме. Яркие цвета только для игровых элементов.
5. **Не только цвет.** Команды и варианты ответов различаются цветом и фигурой или иконкой, чтобы было понятно и дальтоникам, и на плохом проекторе.
6. **Без мусора.** Никаких дефолтных HTML-кнопок, серых таблиц и браузерных стилей. Каждый элемент оформлен.

Дизайн-система

Цвета игры (одинаковые в светлой и тёмной теме, на тёмной фон под ними чуть темнее):

Роль	Цвет	Применение
Команда 1 «Красные»	`#F43F5E`	Цвет команды, канат, трек гонки
Команда 2 «Синие»	`#3B82F6`	То же
Команда 3 «Зелёные»	`#10B981`	То же
Вариант A, треугольник	`#F43F5E`	Викторина
Вариант B, ромб	`#3B82F6`	Викторина
Вариант C, круг	`#F59E0B`	Викторина
Вариант D, квадрат	`#10B981`	Викторина
Акцент / главная кнопка	`#8B5CF6`	Кнопки действий, логотип, выделение
Успех	`#22C55E`	Правильный ответ
Ошибка	`#EF4444`	Неправильный ответ

Роль	Цвет	Применение
Предупреждение	`#F59E0B`	Флаги анти-чита, мало времени

Типографика:

- Интерфейс: шрифт VS Code (`--vscode-font-family`), код — шрифт редактора (`--vscode-editor-font-family`).
- Размеры: 13 px — обычный текст, 16 px — подписи кнопок, 20–24 px — заголовки экранов, 48–64 px — таймер, счёт, код комнаты.
- Цифры таймера и счёта — моноширинные (`font-variant-numeric: tabular-nums`), чтобы не прыгали.

Форма и отступы: сетка 4 px (отступы 8, 12, 16, 24, 32), скругление 12 px у карточек и 10 px у кнопок, тонкие рамки 1 px вместо теней. Главная кнопка высотой не меньше 44 px.

Анимации (Framer Motion):

- Обычные переходы 150–250 мс, `ease-out` .
- Канат двигается пружинной анимацией (`spring`), с лёгким «рывком» при каждом шаге.
- Таблица лидеров: строки плавно меняются местами (`layout` -анимация).
- Победа: конфетти (`canvas-confetti`) и пьедестал на 3 места.
- Последние 5 секунд таймера: цифры пульсируют и становятся оранжевыми.
- Уважать `prefers-reduced-motion` : при нём только мягкие затухания.

Звуки: короткие и негромкие (правильно, ошибка, тик последних секунд, победа). У учеников по умолчанию выключены (класс шумный), у экрана проектора включены. Переключатель всегда виден.

Иконки: `lucide-react` , один набор на весь проект, размер 16 или 20 px.

Управление с клавиатуры: клавиши `1` – `4` выбирают вариант в викторине, `Ctrl+Enter` отправляет решение, `Esc` закрывает диалоги.

Где что показывается в VS Code

- **Боковая панель** (иконка расширения в Activity Bar): вход, лобби, статус игры, свой счёт. Должна хорошо выглядеть при ширине от 280 px.
- **Панель игры** (`WebviewPanel` в области редактора): канат, викторина, гонка, таблицы — всё, чему нужно место.

- **Обычный редактор** для задач с кодом: файл с условием в комментарии сверху, кнопки «Запустить тесты» и «Отправить» над редактором (CodeLens) и в панели статуса.

Экраны ученика

1. **Вход.** Логотип, поле «Код комнаты» (6 крупных ячеек), поле «Имя и фамилия», кнопка «Войти». Если пришёл по ссылке, код уже заполнен.
2. **Лобби.** Крупно своё имя и цвет команды, состав команды аватарками-инициалами, спокойная анимация ожидания и текст «Ждём, пока учитель начнёт игру».
3. **Отсчёт перед стартом.** На весь экран «3, 2, 1, Начали!» с названием игры.
4. **Канат.** Сверху канат на всю ширину с узлом посередине и цветами команд по краям, под ним задача крупным шрифтом, поле ответа и кнопка «Отправить». После ошибки поле на 3 секунды блокируется с круговым индикатором.
5. **Викторина.** Вопрос сверху, таймер-кольцо, 4 большие цветные плитки с фигурами в сетке 2×2. После ответа: «Ответ принят», затем правильно или нет и полученные очки.
6. **Codewars.** Условие и тесты в боковой панели, код в редакторе. Результаты тестов — список с зелёными и красными отметками. После отправки: «Решено! Ты на 4 месте» и позиция в списке.
7. **Гонка.** Трек с фигурками игроков или команд, текущая задача под ним, прогресс «3 из 10».
8. **Найди баг.** Код с подсветкой синтаксиса, строки кликабельны и подсвечиваются при наведении. Для «выбери лучший» — карточки вариантов кода рядом.
9. **Объяснение после задания.** Короткая карточка: правильный ответ и 1–2 предложения почему.
10. **Итоги.** Пьедестал на 3 места, конфетти, своё место и очки, кнопка «Посмотреть решения» (для Codewars).

Экраны учителя

1. **Создание комнаты.** Кнопка «Создать комнату», затем код комнаты крупным шрифтом (удобно показать на проекторе) и кнопка «Скопировать ссылку».
2. **Лобби и команды.** Слева «Без команды», справа колонки команд, имена перетаскиваются (`dnd-kit`). Сверху переключатель «Каждый сам за себя / 2 команды / 3 команды» и кнопка «Распределить случайно».

3. **Выбор игры.** 5 крупных карточек с иконкой, названием и одной строкой описания. Канат неактивен при 3 командах с подсказкой почему.
4. **Задания.** Вкладки «Вручную / По теме (ИИ) / Из банка», список заданий карточками с правкой и удалением, предпросмотр «как увидит ученик».
5. **Настройки игры.** Только нужные для выбранной игры поля, значения по умолчанию уже стоят, кнопка «Начать».
6. **Ход игры.** Слева та же визуализация, что у учеников, справа живая таблица учеников со статусами и флагами. Сверху «Пауза», «Дальше», «Завершить».
7. **Итоги.** Таблица результатов, кнопки «Экспорт CSV», «Сыграть ещё раз», «Новая игра».

Состояния, которые обязаны быть красивыми

- **Пусто:** «Пока никто не подключился. Отправьте ссылку в группу» с кнопкой копирования.
- **Загрузка:** скелетоны, а не голый спиннер.
- **Ошибка сети:** баннер «Соединение потеряно, переподключаемся…» и автоповтор, без потери прогресса.
- **Нет Python:** понятная карточка с кнопкой «Как установить».

Страница для проектора (после MVP)

Веб-страница по ссылке комнаты: огромный код комнаты в лобби, затем канат, трек гонки или таблица лидеров на весь экран, без лишних элементов. Читается с последней парты.

Протокол WebSocket и сервер

Одна комната = один Durable Object. Сервер — единственный источник правды: очки, порядок ответов, позиция каната, таймеры считаются только на нём, клиенты лишь показывают состояние.

Сервер:

- Worker принимает `POST /rooms` (создать комнату, вернуть код и токен учителя) и `GET /rooms/:code/ws` (апгрейд до WebSocket), дальше пересылает соединение в Durable Object комнаты.
- Используется WebSocket Hibernation API, чтобы неактивные комнаты не тратили бесплатный лимит.
- Таймеры раундов через Durable Object Alarms.

- Состояние комнаты хранится в SQLite внутри Durable Object, чтобы пережить перезапуск.
- Все сообщения — JSON с полем `type`, валидируются схемами Zod из `shared` с обеих сторон.
- Тесты сервера на Vitest + `@cloudflare/vitest-pool-workers`.

Сообщения от клиента:

type	Кто	Данные
`join`	Ученик	имя, токен (если переподключение)
`set_teams`	Учитель	режим деления, распределение игроков
`rename_player` / `kick_player`	Учитель	id игрока, новое имя
`start_game`	Учитель	игра, настройки, задания
`pause` / `resume` / `next` / `end_game`	Учитель	—
`submit`	Ученик	id задания, ответ или код + результат тестов
`flag`	Расширение ученика	тип флага, детали
`ping`	Все	—

Сообщения от сервера:

type	Кому	Данные
`joined`	Ученику	id, токен, команда
`lobby`	Всем	список игроков, команды
`game_started`	Всем	игра, настройки
`task`	Всем или команде	задание без правильного ответа, дедлайн
`result`	Ученику	правильно или нет, очки, блокировка до
`state`	Всем	счёт, позиция каната, трек, таблица лидеров

`type`	Кому	Данные
`reveal`	Всем	правильный ответ, объяснение, статистика
`teacher_feed`	Учителю	все отправки (включая неправильные) и флаги
`game_over`	Всем	итоговые места
`error`	Отправителю	код ошибки, текст

Важно: сообщение `task` никогда не содержит правильного ответа для серверных типов заданий. Для заданий с кодом видимые тесты отправляются, потому что проверка локальная.

Публикация расширения

Расширение публикуется бесплатно в VS Code Marketplace и Open VSX автоматически через GitHub Actions при создании релиза.

- Для тестов с одногруппниками: `vsce package` создаёт файл `.vsix`, он ставится через «Extensions → … → Install from VSIX».
- Publisher создаётся на странице управления Marketplace под Microsoft-аккаунтом, его ID указывается в поле `publisher` в `package.json`. ID потом нельзя изменить.
- С 1 декабря 2026 года глобальные Personal Access Tokens в Azure DevOps отключаются, поэтому публикация из CI идёт без токенов: `npx @vscode/vsce publish --oidc` в GitHub Actions с правом `id-token: write`. Вручную можно загрузить `.vsix` на странице управления publisher'а.
- Параллельно публикация в Open VSX (`npx ovsx publish`), чтобы расширение ставилось в VSCodium, Cursor и других форках.
- Иконка расширения в PNG: Marketplace не принимает SVG.
- В README: GIF с игрой в канат, скриншоты, инструкция «Как начать урок за 1 минуту».
- Сервер деплоится через `wrangler deploy` в отдельном GitHub Actions workflow.

Источник: Publishing Extensions — VS Code Docs.

Этапы разработки и критерии готовности

Разработка идёт этапами, каждый заканчивается рабочей версией, которую можно попробовать в классе. Дизайн делается сразу красиво на каждом этапе, а не «потом».

1. **Каркас.** Монорепозиторий, пакеты `extension`, `webview-ui`, `server`, `shared`, дизайн-система (токены цветов, кнопки, карточки, поля), локальный запуск сервера через `wrangler dev`.
2. **Комнаты и лобби.** Создание комнаты, вход по коду и ссылке, имена, переподключение, деление на команды перетаскиванием и случайно.
3. **Викторина.** Полный цикл: задания вручную, таймер, очки, итоги с пьедесталом.
4. **Перетягивание каната.** Серверный порядок ответов, штраф за ошибку, анимация каната.
5. **Codewars и запуск кода.** Локальные тесты Python и JS, сортировка по длине, просмотр решений.
6. **Гонка и «Найди баг».** Включая проверку HTML/CSS по правилам.
7. **Защита от списывания и панель учителя.** Флаги, живая таблица, просмотр решений, экспорт CSV.
8. **Генерация заданий через ИИ и банк задач.**
9. **Локализация** (ru, ky, en) и публикация в Marketplace и Open VSX.
10. **После MVP:** страница для проектора, Docker-образ сервера.

Критерии готовности первой версии:

- [ ] Учитель создаёт комнату и запускает игру меньше чем за 1 минуту.
- [ ] 30 учеников играют одновременно без задержек заметнее 300 мс.
- [ ] Все 5 игр проходятся от лобби до итогов без ошибок.
- [ ] Ученик, потерявший связь, возвращается со своими очками и в свою команду.
- [ ] Интерфейс выглядит аккуратно в светлой и тёмной теме VS Code и в боковой панели шириной 280 px.
- [ ] Новый ученик понимает, что делать, без объяснений учителя.
- [ ] Сервер работает на бесплатном плане Cloudflare, ежемесячные расходы равны нулю.
- [ ] Ядро логики (очки, порядок ответов, сортировка решений) покрыто тестами.